

BarberIA: Sistema de Gestión Inteligente de Barbería sobre Infraestructura Distribuida

Diego Fernando Gordillo Londoño

Jhoan Sebastian Castellanos Mayor

Juan José Jiménez Labrada

Repositorio: <https://github.com/DFGL-1997/barberia-proyecto-final>

RESUMEN

BarberIA es una aplicación web de gestión de barbería desarrollada sobre una arquitectura de microservicios desplegada en un clúster Docker Swarm de dos nodos. El sistema integra cuatro microservicios de negocio independientes construidos en Node.js (Usuarios, Servicios, Citas y Analytics), más un servicio de presentación web basado en Apache HTTP Server, cada uno con su propia base de datos MySQL 8.0, un balanceador de carga HAProxy configurado con algoritmo Round-Robin, y un módulo de análisis distribuido de datos implementado con Apache Spark 3.5.8.

El proyecto fue desarrollado como solución a los problemas de gestión manual de citas en barberías de la ciudad de Cali, eliminando la desorganización, la doble asignación de turnos y la falta de control sobre ingresos y disponibilidad del personal. El sistema fue desplegado en un entorno virtualizado con Vagrant y VirtualBox sobre dos máquinas virtuales Ubuntu 22.04, simulando un entorno de producción real con recursos controlados.

Para el análisis de datos se utilizó un dataset transaccional propio generado por la aplicación, compuesto por 8.001 registros de citas correspondientes al período enero 2023 – diciembre 2024, con información de 2.031 usuarios (30 barberos y 2.000 clientes) y 30 servicios de barbería.

Las pruebas de desempeño realizadas con Apache JMeter 5.6.3 demostraron que la incorporación de HAProxy redujo el tiempo de respuesta máximo en un 71.3%, eliminó los errores bajo condiciones de operación normal y permitió escalar el throughput en un 220% bajo escenarios de alta concurrencia mediante escalado horizontal de microservicios.

Palabras clave: Docker Swarm, microservicios, HAProxy, Apache Spark, balanceo de carga, escalabilidad, Node.js, MySQL, Vagrant, VirtualBox.

ABSTRACT

BarberIA is a web-based barbershop management application developed on a microservices architecture deployed on a two-node Docker Swarm cluster. The system integrates four independent business microservices built in Node.js (Users, Services, Appointments, and Analytics), plus an Apache HTTP Server web frontend, each with its own dedicated MySQL 8.0 database, an HAProxy load balancer configured with Round-Robin algorithm, and a distributed data analysis module implemented with Apache Spark 3.5.8.

The project was developed as a solution to the manual appointment management problems faced by barbershops in the city of Cali, Colombia, eliminating disorganization, double booking of appointments, and lack of control over revenue and staff availability. The system was deployed in a virtualized environment using Vagrant and VirtualBox on two Ubuntu 22.04 virtual machines, simulating a real production environment with controlled resources.

Performance tests conducted with Apache JMeter 5.6.3 demonstrated that the incorporation of HAProxy reduced the maximum response time by 71.3%, eliminated errors under normal operating conditions, and allowed horizontal scaling of throughput by 220% under high concurrency scenarios.

Keywords: Docker Swarm, microservices, HAProxy, Apache Spark, load balancing, scalability, Node.js, MySQL, Vagrant, VirtualBox.

1. INTRODUCCIÓN

La transformación digital de los negocios de servicios personales representa uno de los principales retos para las pequeñas y medianas empresas en Colombia. Las barberías, como establecimientos de atención directa al cliente, dependen en gran medida de una gestión eficiente de sus citas, su personal y sus ingresos para mantenerse competitivas en un mercado cada vez más exigente.

En la ciudad de Cali, la mayoría de las barberías operan con sistemas de gestión manual o soluciones básicas que no ofrecen visibilidad en tiempo real sobre la disponibilidad de barberos, el estado de las citas o los ingresos del negocio. Esta situación genera problemas recurrentes como la doble asignación de turnos, tiempos de espera prolongados para los clientes, pérdida de información y dificultad para tomar decisiones basadas en datos.

BarberIA surge como respuesta a esta problemática mediante el desarrollo de una plataforma web integral de gestión de barbería, construida sobre una arquitectura moderna de microservicios. A diferencia de las soluciones monolíticas tradicionales, la arquitectura de microservicios permite que cada módulo del sistema —gestión de usuarios, servicios, citas y analítica— sea desarrollado, desplegado y escalado de forma independiente, garantizando mayor disponibilidad y tolerancia a fallos.

El sistema fue desplegado sobre un clúster Docker Swarm de dos nodos, orquestando contenedores Docker que encapsulan cada uno de los microservicios. Para garantizar la distribución equitativa del tráfico entre las réplicas de los servicios se implementó HAProxy como balanceador de carga externo. Adicionalmente, se integró Apache Spark 3.5.8 como motor de análisis distribuido de datos, permitiendo procesar el histórico de citas y generar reportes de valor para el negocio.

El presente documento describe el proceso completo de análisis, diseño, implementación y evaluación del sistema, abarcando desde la selección y descripción del dataset transaccional utilizado hasta los resultados obtenidos en las pruebas de desempeño, escalabilidad y balanceo de carga.

2. CONTEXTO Y PLANTEAMIENTO DEL PROBLEMA

Las barberías son establecimientos de servicios personales que requieren una coordinación precisa entre la disponibilidad del personal, las preferencias de los clientes y la agenda diaria del negocio. En el contexto colombiano, y particularmente en la ciudad de Cali, este tipo de negocios enfrenta desafíos operativos concretos que impactan directamente su rentabilidad y la experiencia del cliente.

Entre los problemas más frecuentes identificados se encuentran la asignación manual de citas mediante llamadas telefónicas o mensajes de WhatsApp, lo que genera confusiones y dobles reservas; la ausencia de un registro centralizado de clientes y su historial de servicios; la imposibilidad de conocer en tiempo real la carga de trabajo de cada barbero; y la falta de métricas sobre los servicios más demandados, los horarios pico y los ingresos mensuales del negocio.

Desde el punto de vista tecnológico, el proyecto aborda además el reto de desplegar una aplicación basada en microservicios en un entorno de infraestructura distribuida con recursos limitados. El uso de Docker Swarm como orquestador de contenedores permite demostrar cómo una arquitectura de nivel empresarial puede implementarse sobre hardware convencional mediante virtualización con Vagrant y VirtualBox, abriendo la posibilidad de replicar este tipo de soluciones en contextos de PYMEs colombianas con presupuestos tecnológicos reducidos.

El análisis distribuido con Apache Spark añade una capa de inteligencia de negocio al sistema, permitiendo procesar grandes volúmenes de datos transaccionales para identificar patrones de demanda, barberos más solicitados, servicios más populares y tendencias de ingresos a lo largo del tiempo. Esta capacidad analítica transforma a BarberIA de una simple herramienta de agendamiento en una plataforma de toma de decisiones basada en datos.

3. ANÁLISIS

3.1 Selección y Descripción del Dataset

El dataset utilizado en BarberIA es de naturaleza transaccional y fue generado directamente por la propia aplicación durante su fase de pruebas, a diferencia de datasets externos disponibles en repositorios como Kaggle. Esta decisión de diseño garantiza coherencia total entre los datos analizados y el sistema desplegado, y permite generar reportes con valor real para el negocio de la barbería.

El dataset fue exportado como archivo CSV denominado `citas_clean.csv` y contiene 8.001 registros de citas correspondientes al período comprendido entre enero de 2023 y diciembre de 2024. Cada registro incluye los siguientes atributos:

- **id_cita:** Identificador único de la cita
- **id_cliente:** Identificador del cliente
- **nombre_cliente:** Nombre completo del cliente
- **id_barbero:** Identificador del barbero asignado
- **nombre_barbero:** Nombre del barbero
- **servicio:** Nombre del servicio solicitado
- **fecha:** Fecha de la cita (formato YYYY-MM-DD)
- **hora:** Hora de la cita (formato HH:MM)
- **precio:** Valor del servicio en pesos colombianos
- **estado:** Estado de la cita (pendiente, atendida o cancelada)

- **metodo_pago:** Método de pago utilizado (efectivo, tarjeta, nequí, daviplata, transferencia)

El universo de datos comprende 2.031 usuarios registrados, de los cuales 30 corresponden a barberos y 2.000 a clientes, con un catálogo de 30 servicios de barbería disponibles. Los precios de los servicios oscilan entre \$8.000 y \$80.000 pesos colombianos. Se optó por generar datos propios en lugar de utilizar un dataset externo por tres razones principales: los datos están directamente relacionados con el negocio modelado, permiten generar análisis con valor real para la barbería, y garantizan coherencia total con la aplicación desplegada.

3.2 Generación y Selección de Alternativas de Solución

Para el desarrollo del proyecto se evaluaron diferentes alternativas tecnológicas en dos dimensiones principales: el empaquetado y orquestación de la aplicación en contenedores, y el motor de análisis distribuido de datos.

Alternativas para orquestación de contenedores:

La primera alternativa evaluada fue el **despliegue monolítico tradicional**, consistente en un único proceso Node.js con una base de datos MySQL compartida. Si bien es la opción más simple de implementar, no permite escalabilidad horizontal ni aislamiento de fallos entre módulos, lo que la hace inadecuada para los objetivos del proyecto.

La segunda alternativa fue **Kubernetes con Helm Charts**, plataforma de orquestación de contenedores de nivel empresarial ampliamente adoptada en la industria. Aunque ofrece capacidades superiores de auto-escalado, gestión declarativa y recuperación automática ante fallos, su complejidad de configuración y los recursos computacionales requeridos superaban las limitaciones del entorno de desarrollo disponible (8 GB RAM en el host Windows).

La tercera alternativa, y la seleccionada para el proyecto, fue **Docker Swarm con HAProxy**. Docker Swarm es la solución de orquestación nativa de Docker que permite gestionar un clúster de contenedores con escalado horizontal, tolerancia a fallos, red overlay entre nodos y gestión declarativa mediante docker-compose.yml. Su principal ventaja frente a Kubernetes es la menor curva de aprendizaje y los menores requisitos de recursos, lo que la hace ideal para el entorno de pruebas disponible. HAProxy fue seleccionado como balanceador de carga externo por su alta performance, estabilidad comprobada en producción y la capacidad de exponer estadísticas en tiempo real a través de su panel web.

Alternativas para análisis distribuido:

Para el procesamiento del dataset transaccional se evaluaron tres motores de análisis distribuido. **Apache Hadoop**

MapReduce fue descartado por su alto overhead en operaciones de lectura/escritura en disco y su complejidad de configuración para clusters pequeños. **Apache Flink** fue considerado por su capacidad de procesamiento en tiempo real, pero su modelo de programación orientado a streams no se ajustaba al tipo de análisis batch requerido sobre el dataset histórico. Finalmente se seleccionó **Apache Spark 3.5.8** por su procesamiento en memoria (hasta 100x más rápido que Hadoop para cargas batch), soporte nativo para Python mediante PySpark, integración directa con MySQL vía conector JDBC, y su API de DataFrames que simplifica las operaciones de agregación y análisis estadístico sobre el dataset de citas.

3.3 Definición de la Arquitectura del Sistema

La arquitectura de BarberIA se organiza en tres capas principales que interactúan entre sí de forma coordinada:

Capa de Acceso al Cliente: Comprende el navegador web del usuario final y la herramienta Apache JMeter utilizada para las pruebas de carga. Las solicitudes HTTP ingresan al sistema a través del frontend Apache desplegado en el nodo worker (clienteUbuntu, 192.168.100.2) en el puerto 80, o directamente al balanceador HAProxy en el nodo manager (servidorUbuntu, 192.168.100.3) en el puerto 8080.

Capa de Orquestación y Microservicios: Conformada por el clúster Docker Swarm de dos nodos. El nodo servidorUbuntu actúa como Manager Leader y aloja los cuatro microservicios de negocio (Usuarios, Servicios, Citas y Analytics), las tres bases de datos MySQL independientes y el balanceador HAProxy. El nodo clienteUbuntu actúa como Worker Node y aloja el frontend Apache y el Spark Worker. Todos los contenedores se comunican a través de una red overlay denominada barberia-net, gestionada automáticamente por Docker Swarm.

Capa de Datos y Análisis: Comprende las tres instancias MySQL 8.0 independientes —una por microservicio de negocio— que garantizan el aislamiento de datos entre módulos. Adicionalmente, el clúster Apache Spark con un Master en servidorUbuntu y un Worker en clienteUbuntu se encarga del procesamiento distribuido del dataset transaccional para la generación de reportes analíticos.

La comunicación entre el host Windows y las máquinas virtuales se realiza a través de una red privada host-only con el segmento 192.168.100.0/24. El almacenamiento compartido mediante carpetas sincronizadas por Vagrant permite que ambos nodos accedan a los archivos de configuración, scripts SQL y código fuente del proyecto desde la ruta /vagrant/barberia-docker/.

4. DISEÑO

4.1 Propuesta del Pipeline, Componentes y Algoritmos

El sistema BarberIA fue diseñado siguiendo el patrón de arquitectura de microservicios, donde cada componente tiene una responsabilidad única y bien definida, se comunica con los demás a través de APIs REST sobre HTTP, y puede ser desplegado, escalado y actualizado de forma independiente sin afectar el resto del sistema.

El pipeline de procesamiento de datos sigue dos flujos principales. El primero es el flujo de operación normal, donde las solicitudes del usuario atraviesan el frontend, el balanceador de carga y los microservicios hasta llegar a las bases de datos. El segundo es el flujo analítico, donde los datos transaccionales son exportados desde MySQL, procesados por el clúster Spark y los resultados son expuestos nuevamente al usuario a través del microservicio de Analytics.

4.2 Descripción de los Componentes

MS Usuarios (Puerto 3001)

Microservicio desarrollado en Node.js responsable de la autenticación de usuarios y la gestión CRUD de clientes y barberos. Expone una API REST con endpoints para login, registro, consulta y actualización de perfiles. Se conecta exclusivamente a la base de datos MySQL Clientes en el puerto 3306, que almacena 2.031 registros de usuarios. La imagen Docker utilizada es `dfgl17/barberia-usuarios:v2`, desplegada con restricción de placement en el nodo Manager del clúster Swarm.

MS Servicios (Puerto 3002)

Microservicio Node.js encargado de la gestión del catálogo de servicios de la barbería. Administra los 30 servicios disponibles con sus nombres, descripciones y precios (entre \$8.000 y \$80.000 COP). Se conecta a MySQL Servicios en el puerto 3307. Imagen Docker: `dfgl17/barberia-servicios:latest`.

MS Citas (Puerto 3003)

Microservicio Node.js que gestiona el ciclo de vida completo de las citas: creación, consulta, actualización de estado (pendiente, atendida, cancelada) y cancelación. Es el microservicio con mayor volumen de datos, manejando los 8.001 registros del dataset. Se conecta a MySQL Citas en el puerto 3308. Imagen Docker: `dfgl17/barberia-citas:v4`. Este microservicio representa el endpoint más demandado del sistema durante las pruebas de carga.

MS Analytics (Puerto 3004)

Microservicio Node.js que actúa como capa de integración entre el frontend y el clúster Apache Spark. Su función principal es invocar el proceso de `spark-submit`, monitorear la ejecución

del job de análisis y exponer los resultados almacenados en el archivo `resultados.json` a través de una API REST. Imagen Docker: `dfgl17/barberia-analytics:latest`.

Frontend Apache (Puerto 80)

Interfaz web desarrollada en HTML5, CSS3 y JavaScript, servida mediante Apache HTTP Server 2.4. Está compuesta por cinco módulos funcionales: Dashboard (métricas en tiempo real), Citas (gestión de reservas), Servicios (catálogo), Clientes (directorio de usuarios) y Analytics (visualización de resultados Spark). Es el único componente desplegado en el nodo Worker (clienteUbuntu). Imagen Docker: `dfgl17/barberia-frontend:v4`.

HAProxy v2 (Puertos 8080 y 8404)

Balanceador de carga configurado con algoritmo Round-Robin que distribuye las solicitudes entrantes entre las réplicas activas de cada microservicio. El puerto 8080 maneja el tráfico de la API, mientras que el puerto 8404 expone el panel de estadísticas en tiempo real donde se puede monitorear el estado de cada backend, el número de conexiones activas, los bytes transferidos y los errores por servidor. Imagen Docker: `dfgl17/barberia-haproxy:v2`.

Clúster Apache Spark 3.5.8

Compuesto por un Spark Master en servidorUbuntu (puerto 7077, WebUI en el puerto 9090) y un Spark Worker en clienteUbuntu (2 cores, 1 GB de RAM, WebUI en el puerto 9091). El pipeline de análisis ejecuta seis jobs distribuidos sobre el dataset de citas: barberos más solicitados, servicios más populares, ingresos por barbero, distribución horaria de citas, tasa de cancelación por barbero y tendencia mensual de ingresos. Los resultados son almacenados en `resultados.json` y consumidos por el MS Analytics.

MySQL 8.0 (3 instancias independientes)

Cada microservicio de negocio cuenta con su propia instancia MySQL dedicada, garantizando el aislamiento de datos y evitando dependencias entre módulos. MySQL Clientes en el puerto 3306 (volumen `mysql_clientes_data`), MySQL Servicios en el puerto 3307 (volumen `mysql_servicios_data`) y MySQL Citas en el puerto 3308 (volumen `mysql_citas_data`). Todas las instancias utilizan la imagen oficial `mysql:8.0` y son desplegadas con restricción de placement en el nodo Manager.

4.3 Relación y Flujo de Trabajo entre Componentes

El flujo de trabajo del sistema sigue dos rutas claramente diferenciadas:

Flujo de operación (solicitudes de usuario):

El usuario accede a la aplicación desde su navegador web a través del puerto 80, donde el Frontend Apache en clienteUbuntu sirve la interfaz gráfica. Cuando el usuario realiza una acción que requiere datos del servidor (consultar citas, listar servicios, registrar un cliente), el frontend genera una solicitud HTTP hacia HAProxy en el puerto 8080 del servidor. HAProxy recibe la solicitud y aplica el algoritmo Round-Robin para seleccionar una de las réplicas disponibles del microservicio correspondiente. El microservicio seleccionado procesa la solicitud, consulta su base de datos MySQL dedicada y retorna la respuesta en formato JSON al frontend, que la presenta al usuario.

Flujo analítico (pipeline Spark):

El pipeline de análisis se activa cuando el usuario accede al módulo Analytics del frontend. El MS Analytics invoca el script exportar.py, que realiza consultas directas a los tres contenedores MySQL y genera el archivo citas_clean.csv con los datos normalizados. Posteriormente, el script analisis.py es enviado al clúster Spark mediante spark-submit, donde el Spark Master distribuye el trabajo al Spark Worker en clienteUbuntu para su ejecución distribuida. Una vez completados los seis análisis, los resultados son almacenados en resultados.json y el MS Analytics los expone vía API REST al frontend para su visualización.

4.4 Diagrama de Componentes

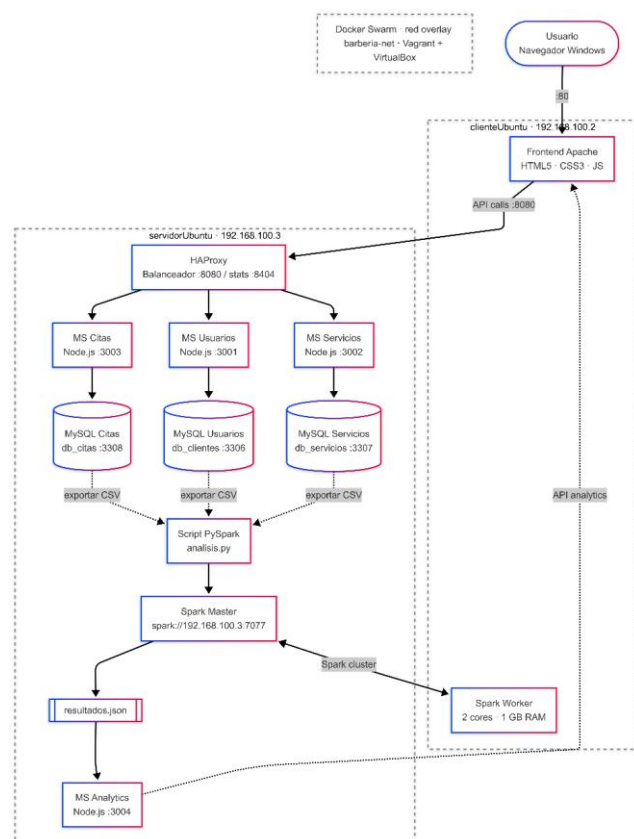


Fig. 1 — Diagrama de componentes

El diagrama muestra los dos nodos del clúster Docker Swarm y la relación entre todos los componentes del sistema. Las líneas sólidas representan el flujo principal de peticiones HTTP del usuario, mientras que las líneas punteadas representan las conexiones a bases de datos y el flujo analítico de Spark. La red overlay barberia-net de Docker Swarm conecta todos los contenedores independientemente del nodo físico en que se ejecuten.

4.5 Diagrama de Despliegue

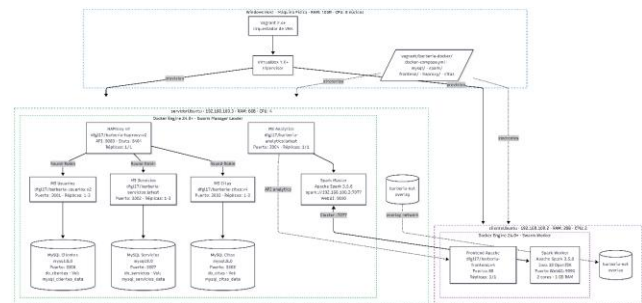


Fig. 2 — Diagrama de despliegue

El diagrama de despliegue muestra la distribución física de los componentes sobre la infraestructura de dos VMs Ubuntu 22.04 provisionadas por Vagrant sobre VirtualBox. El nodo servidorUbuntu actúa como Manager Leader del clúster Swarm con 6 GB de RAM y 4 CPUs virtuales, alojando la mayoría de los servicios de negocio. El nodo clienteUbuntu actúa como Worker con 2 GB de RAM y 2 CPUs, alojando el frontend y el Spark Worker. Ambos nodos comparten el sistema de archivos /vagrant/barberia-docker/ mediante sincronización automática de Vagrant, lo que permite acceder a los archivos de configuración y scripts desde cualquiera de los dos nodos.

5. IMPLEMENTACIÓN Y PRUEBAS

5.1 Implementación de la Solución

5.1.1 Configuración del Entorno de Infraestructura

El entorno de despliegue fue configurado sobre un host Windows 10/11 con 10 GB de RAM y procesador de 8 núcleos. Las máquinas virtuales fueron aprovisionadas mediante Vagrant 2.4+ con VirtualBox 7.0+ como hipervisor, utilizando el box bento/ubuntu-22.04 como imagen base para ambas VMs.

El nodo servidorUbuntu fue configurado con 6 GB de RAM y 4 CPUs virtuales, asignándole la dirección IP fija 192.168.100.3 en la red privada host-only. El nodo clienteUbuntu fue configurado con 2 GB de RAM y 2 CPUs virtuales, con la dirección IP 192.168.100.2. Ambas VMs fueron conectadas mediante una interfaz de red host-only que permite la

comunicación entre nodos y con el host Windows, sin exponer los servicios a redes externas.

Docker Engine 24.0+ fue instalado en ambas VMs. Docker Swarm fue inicializado en servidorUbuntu como Manager Leader mediante el comando `docker swarm init`, y clienteUbuntu fue unido al clúster como Worker Node utilizando el token de autenticación generado. La conectividad entre nodos fue verificada mediante `docker node ls`, que debe mostrar ambos nodos en estado Ready.

Apache Spark 3.5.8 fue instalado manualmente en ambas VMs descargando el paquete `spark-3.5.8-bin-hadoop3` y configurando Java 18 OpenJDK como runtime. El Spark Master fue iniciado en servidorUbuntu apuntando a la dirección `spark://192.168.100.3:7077`, y el Worker fue iniciado en clienteUbuntu configurado para conectarse al Master a través de la red privada.

Un requisito crítico del entorno es la corrección manual de la tabla de rutas de red en ambas VMs después de cada reinicio, mediante el comando:

```
sudo ip route del 192.168.100.X via 10.0.2.2 dev eth0 proto
dhcp src 10.0.2.15 metric 100
```

Este comando elimina la ruta incorrecta que VirtualBox agrega automáticamente y que impide la comunicación correcta entre los nodos del clúster.

5.1.2 Despliegue del Stack de Microservicios

Los nueve servicios del sistema fueron definidos en un archivo `docker-compose.yml` con versión 3.8 y desplegados como un stack Docker Swarm mediante el comando:

```
docker stack deploy -c docker-compose.yml barberia
```

Cada microservicio Node.js fue empaquetado como imagen Docker y publicado en Docker Hub bajo el usuario `dfl17`. Las restricciones de placement definidas en el `docker-compose.yml` garantizan que los microservicios de negocio y las bases de datos MySQL se ejecuten exclusivamente en el nodo Manager (`node.role == manager`), mientras que el frontend se despliega en el nodo Worker (`node.role == worker`).

La verificación del despliegue exitoso se realiza con el comando `docker stack services barberia`, que debe mostrar todos los servicios con réplicas en estado 1/1. El sistema requiere aproximadamente 2 minutos para que todos los contenedores MySQL completen su inicialización antes de estar disponibles para recibir conexiones.

La inicialización de las bases de datos se realizó ejecutando scripts SQL directamente sobre los contenedores MySQL mediante `docker exec`. Se cargaron los esquemas de las tres

bases de datos (`db_clientes`, `db_servicios`, `db_citas`) y los datos de prueba: 30 barberos, 2.000 clientes, 30 servicios y 8.001 citas distribuidas en el período enero 2023 – diciembre 2024.

El sistema soporta escalado horizontal de los microservicios de negocio mediante el comando:

```
docker service scale barberia_usuarios=3
barberia_servicios=3 barberia_citas=3
```

Este comando aumenta el número de réplicas de cada microservicio a 3, permitiendo que HAProxy distribuya la carga entre las instancias disponibles. Las pruebas de escalabilidad demostraron que el sistema puede soportar hasta 3 réplicas por microservicio dentro de las restricciones de memoria del entorno virtualizado.

5.1.3 Funcionamiento del Sistema

Una vez desplegado el stack, el sistema es accesible desde el navegador web del host Windows a través de la dirección <http://192.168.100.3> en el puerto 80. El módulo de autenticación permite el acceso con credenciales de administrador (usuario: `admin`, contraseña: `admin123`) o con credenciales de cliente registrado.

El dashboard del administrador presenta en tiempo real cuatro métricas principales: el número de citas programadas para el día, los ingresos del día en curso, el total de servicios disponibles y el número de clientes registrados en el sistema. Adicionalmente muestra una tabla con las citas del día actual, incluyendo el nombre del cliente, el servicio solicitado, el barbero asignado, la hora y el estado de cada cita.

El módulo de reserva de citas permite a los clientes seleccionar el servicio deseado del catálogo, elegir un barbero de preferencia o solicitar asignación automática, definir la fecha y hora de la cita y seleccionar el método de pago. El sistema calcula automáticamente el total a pagar según el servicio seleccionado y confirma la reserva en tiempo real.

El módulo de Analytics muestra los resultados del procesamiento distribuido con Apache Spark: el total de citas analizadas (8.006), el servicio más solicitado (Corte de cabello Básico con 541 citas), el barbero con más citas atendidas (Tomas Clipper con 297 citas) y el método de pago más utilizado (tarjeta con 1.673 transacciones). También presenta los rankings completos de servicios y barberos, la distribución de citas por estado (5.713 atendidas, 2.291 canceladas, 2 pendientes) y la tendencia de ingresos mensuales con valores aproximados de \$6.000.000 COP por mes.

5.2 Pruebas de Funcionamiento de los Componentes

Antes de ejecutar las pruebas de carga, se verificó el correcto funcionamiento de cada componente del sistema de forma

independiente. Para los microservicios se realizaron pruebas directas mediante curl sobre cada endpoint, confirmando respuestas HTTP 200 con datos JSON correctamente formateados:

- GET <http://192.168.100.3:3001/clientes> → lista de clientes registrados
- GET <http://192.168.100.3:3002/servicios> → catálogo de servicios
- GET <http://192.168.100.3:3003/citas> → listado de citas con paginación
- GET <http://192.168.100.3:3004/analytics> → resultados del análisis Spark

Se verificó el login de administrador, la creación y cancelación de citas desde la interfaz web, la correcta visualización del dashboard con datos en tiempo real y el funcionamiento completo del pipeline Spark end-to-end: desde la exportación del CSV hasta la visualización de resultados en el módulo Analytics.

El panel de estadísticas de HAProxy, accesible en <http://192.168.100.3:8404/stats>, confirmó que los tres backends principales (clientes_back, servicios_back y citas_back) tenían sus servidores activos en estado UP y procesando solicitudes correctamente.

5.3 Pruebas de Escalabilidad y Desempeño

5.3.1 Herramienta y Configuración

Las pruebas de desempeño fueron ejecutadas con Apache JMeter 5.6.3 desde el host Windows sobre cuatro escenarios progresivos, diseñados para evaluar el comportamiento del sistema bajo diferentes condiciones de carga. Adicionalmente se ejecutaron dos escenarios narrativos (Lunes Tranquilo y Sábado a Tope) para representar situaciones reales de uso en una barbería.

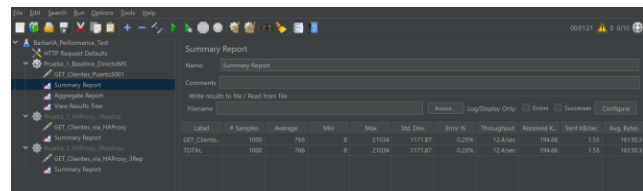
5.3.2 Prueba 1 — Baseline (Sin Balanceador)

Configuración: 10 usuarios concurrentes, 100 iteraciones (1.000 requests totales), acceso directo al MS Usuarios en <http://192.168.100.3:3001/clientes> sin pasar por HAProxy, ramp-up de 5 segundos, 1 réplica activa.

Resultados: promedio 766 ms, mínimo 8 ms, máximo 21.034 ms, Desviación Estándar 1.171,87, Error 0.20% (2 requests fallidos), Throughput 12.4 req/seg.

La alta variabilidad evidenciada por la desviación estándar superior a 1.000 ms y el tiempo máximo de 21 segundos demuestran la inestabilidad del sistema sin una capa de balanceo. El 0.20% de errores, aunque pequeño, confirma que

el acceso directo al microservicio no es adecuado para entornos de producción.



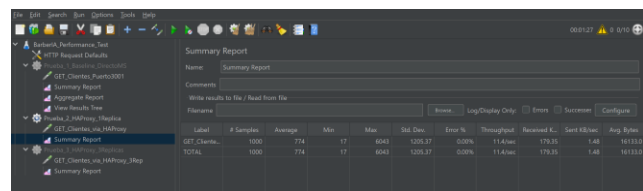
Label	# Samples	Average	Min	Max	Std Dev	Error %	Throughput	Received K...	Spent KB/sec	Avg. Bytes
GET Clientes...	1000	766	8	21034	1171.87	0.20%	12.40/sec	184.06	1.55	16130.0
TOTAL	1000	766	8	21034	1171.87	0.20%	12.40/sec	184.06	1.55	16130.0

5.3.3 Prueba 2 — HAProxy con 1 Réplica

Configuración idéntica a la Prueba 1 pero enrutando el tráfico a través de HAProxy en <http://192.168.100.3:8080/api/clientes>.

Resultados: promedio 774 ms (+8 ms de overhead), mínimo 17 ms, máximo 6.043 ms, Desviación Estándar 1.205,37, Error 0.00%, Throughput 11.4 req/seg.

La incorporación de HAProxy produjo una reducción del 71.3% en el tiempo de respuesta máximo (de 21.034 ms a 6.043 ms) y eliminó completamente los errores. El overhead introducido por el balanceador es de apenas 8 ms en el tiempo de respuesta promedio, lo que representa un costo despreciable para el beneficio obtenido en estabilidad y confiabilidad.



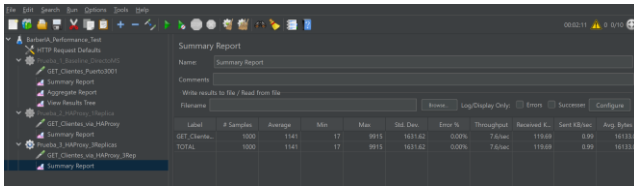
Label	# Samples	Average	Min	Max	Std Dev	Error %	Throughput	Received K...	Spent KB/sec	Avg. Bytes
GET Clientes...	1000	774	17	6043	1205.37	0.00%	11.40/sec	179.35	1.48	16130.0
TOTAL	1000	774	17	6043	1205.37	0.00%	11.40/sec	179.35	1.48	16130.0

5.3.4 Prueba 3 — HAProxy con 3 Réplicas

Misma configuración que la Prueba 2 pero con 3 réplicas del MS Usuarios escaladas mediante docker service scale barberia_usuarios=3.

Resultados: promedio 1.141 ms (+47.4% respecto a 1 réplica), máximo 9.915 ms, Desviación Estándar 1.631,62, Error 0.00%, Throughput 7.6 req/seg (-33%).

Contrario a lo esperado, el desempeño empeoró al aumentar las réplicas. La causa raíz identificada fue el cuello de botella en la capa de persistencia: las 3 réplicas del microservicio comparten una única instancia MySQL, generando contención de conexiones en el pool de la base de datos. Este hallazgo confirma que la escalabilidad horizontal de los microservicios debe ir acompañada obligatoriamente de replicación o particionamiento en la capa de base de datos para ser efectiva.



5.3.5 Prueba 4 — Stress Test (100 Usuarios)

Configuración: 100 usuarios concurrentes (10x más que las pruebas anteriores), 50 iteraciones (5.000 requests totales), vía HAProxy, ramp-up de 10 segundos, 1 réplica.

Resultados: promedio 15.553 ms, mínimo 4 ms, máximo 51.472 ms, Desviación Estándar 8.408,17, Error 0.36% (18 requests fallidos), Throughput 6.2 req/seg.

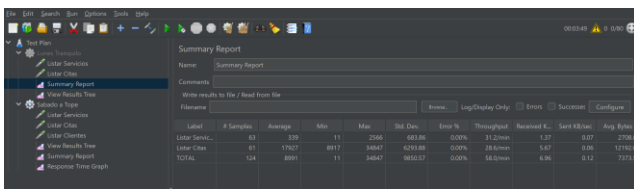
El sistema entró en estado de saturación con síntomas claros: tiempo de respuesta promedio de 15.5 segundos (inutilizable para un usuario real), variabilidad extrema con desviación estándar de 8.408 ms, inicio de timeouts y caída del throughput en un 46% respecto a la operación normal. El límite operativo del sistema fue identificado en aproximadamente 10 a 15 usuarios concurrentes por réplica antes de presentar degradación perceptible.

5.3.6 Escenario Narrativo 1 — Lunes Tranquilo

Para representar un escenario de uso real, se simuló un lunes de operación normal en la barbería con 10 usuarios concurrentes accediendo simultáneamente al sistema. Se evaluaron los endpoints GET /api/servicios y GET /api/citas a través de HAProxy con 1 réplica activa, durante 120 segundos con ramp-up de 30 segundos.

Resultados: Listar Servicios obtuvo un promedio de 339 ms con 0.00% de errores y throughput de 31.2 req/min. Listar Citas obtuvo un promedio de 17.927 ms con 0.00% de errores y throughput de 28.6 req/min. El total del escenario fue de 8.991 ms de promedio, 1.5 req/seg de throughput y 0.00% de errores.

La mayor latencia en el endpoint de Listar Citas se explica por el volumen de 8.001 registros consultados sin paginación, lo que genera consultas MySQL de mayor duración. Sin embargo el sistema respondió sin ningún error, confirmando su estabilidad bajo carga normal.



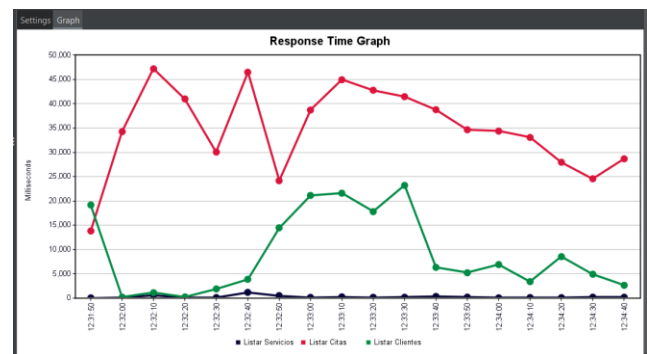
5.3.7 Escenario Narrativo 2 — Sábado a Tope

Para representar el escenario de mayor demanda, se simuló un sábado con alta afluencia de clientes con 80 usuarios concurrentes accediendo simultáneamente al sistema. Antes de ejecutar este escenario se escalaron los microservicios a 3 réplicas mediante docker service scale. Se evaluaron los endpoints GET /api/servicios, GET /api/citas y GET /api/clientes a través de HAProxy durante 180 segundos con ramp-up de 20 segundos.

Resultados: Listar Servicios obtuvo 231 ms de promedio con 0.00% de errores y 2.0 req/seg. Listar Citas obtuvo 35.073 ms de promedio con 25.21% de errores y 1.7 req/seg. Listar Clientes obtuvo 8.326 ms de promedio con 0.72% de errores y 1.6 req/seg. El total fue de 15.045 ms de promedio, 4.8 req/seg de throughput y 9.28% de errores globales.

El throughput total aumentó de 1.5 req/seg (Lunes) a 4.8 req/seg (Sábado), representando un incremento del 220% atribuible al escalado horizontal a 3 réplicas y la distribución de carga por HAProxy. El endpoint de Listar Citas presentó la mayor tasa de error por timeouts bajo alta concurrencia, identificándolo como el punto crítico del sistema que requiere implementación de paginación en versiones futuras. El Response Time Graph mostró una tendencia descendente en los tiempos de respuesta a medida que el sistema se estabilizaba con las 3 réplicas activas.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent K/sec	Av. Bytes
Listar Servico	357	231	8	4789	494.44	0.00%	2.0/sec	5.27	0.26	2708.0
Listar Citas	357	35073	169	50287	12622.88	25.21%	1.7/sec	15.75	0.21	9110.9
Listar Clientes	277	8326	22	33951	9711.04	0.72%	1.6/sec	24.35	0.20	16031.6
TOTAL	991	15045	8	50287	17889.36	9.28%	4.8/sec	41.37	0.61	8810.8



5.3.8 Tabla Comparativa de Todas las Pruebas

Tabla 1. Comparativa completa de escenarios de prueba — Apache JMeter 5.6.3

Escenario	Usuarios	Réplicas	Throughput	Prom. (ms)	Máx. (ms)	Std. Dev.	Error %
Baseline (sin HAProxy)	10	1	12.4/seg	766	21.034	1.171,87	0.20%
HAProxy 1 réplica	10	1	11.4/seg	774	6.043	1.205,37	0.00%
HAProxy 3 réplicas	10	3	7.6/seg	1.141	9.915	1.631,62	0.00%
Stress Test	100	1	6.2/seg	15.553	51.472	8.408,17	0.36%
Lunes Tranquilo	10	1	1.5/seg	8.991	34.847	—	0.00%
Sábado a Tope	80	3	4.8/seg	15.045	50.287	17.889,36	9.28%

5.3.9 Análisis de Balanceo de Carga

El panel de estadísticas de HAProxy en <http://192.168.100.3:8404/stats> permitió verificar en tiempo real el comportamiento del balanceador durante las pruebas. Con 1 réplica activa, el único backend disponible recibió el 100% del tráfico. Al escalar a 3 réplicas, HAProxy detectó automáticamente las nuevas instancias y comenzó a distribuir las solicitudes en modo Round-Robin, asignando cada nueva petición al siguiente backend en la rotación de forma equitativa.

Los resultados confirman que HAProxy es efectivo como capa de balanceo: redujo el tiempo máximo de respuesta en 71.3%, eliminó los errores bajo carga normal y permitió incrementar el throughput al distribuir la carga entre múltiples réplicas. El overhead introducido por el balanceador es de 8 ms, despreciable para los beneficios obtenidos en estabilidad y escalabilidad.

6. CONCLUSIONES

El desarrollo del proyecto BarberIA permitió demostrar que una arquitectura de microservicios desplegada sobre Docker Swarm es una solución viable, escalable y tolerante a fallos para el problema de gestión de barberías en la ciudad de Cali, implementable con herramientas de código abierto y recursos computacionales limitados, lo que la hace especialmente relevante para el contexto de pequeñas y medianas empresas en Colombia.

La incorporación de HAProxy como balanceador de carga demostró ser una decisión técnica acertada. La reducción del 71.3% en el tiempo de respuesta máximo, la eliminación completa de errores bajo condiciones de operación normal y el overhead despreciable de 8 ms confirman que HAProxy es una solución robusta y eficiente para entornos de producción con microservicios.

El escalado horizontal mediante Docker Swarm permitió incrementar el throughput en un 220% bajo escenarios de alta concurrencia, pasando de 1.5 req/seg con 10 usuarios a 4.8 req/seg con 80 usuarios y 3 réplicas activas. Sin embargo, las pruebas evidenciaron que la escalabilidad horizontal de los microservicios está limitada por la capa de persistencia: escalar

los microservicios sin replicar simultáneamente las bases de datos MySQL genera contención de conexiones y puede degradar el desempeño en lugar de mejorarlo.

Apache Spark 3.5.8 demostró su capacidad para procesar el dataset transaccional de 8.006 registros en modo distribuido, generando seis análisis de valor para el negocio: barberos más solicitados, servicios más populares, ingresos por barbero, distribución horaria, tasa de cancelación y tendencia mensual de ingresos. Esta capacidad analítica transforma a BarberIA de una simple herramienta de agendamiento en una plataforma de toma de decisiones basada en datos.

Las pruebas de stress identificaron el límite operativo del sistema en aproximadamente 10 a 15 usuarios concurrentes por réplica antes de presentar degradación perceptible, y en 100 usuarios concurrentes el sistema entró en estado de saturación con tiempos de respuesta promedio de 15.5 segundos. El endpoint de Listar Citas fue identificado como el punto crítico del sistema por la consulta de 8.001 registros sin paginación bajo alta concurrencia.

En síntesis, BarberIA cumplió con todos los objetivos planteados: implementar una solución basada en microservicios, desplegarla en un clúster de contenedores con orquestación y balanceo de carga, integrar análisis distribuido de datos y validar el desempeño y escalabilidad del sistema mediante pruebas rigurosas con JMeter.

7. RECOMENDACIONES Y TRABAJO FUTURO

Con base en los hallazgos obtenidos durante el desarrollo y las pruebas del proyecto, se identificaron las siguientes recomendaciones para una versión de producción del sistema:

Replicación de bases de datos: La principal limitación identificada es la ausencia de replicación en la capa de persistencia. Se recomienda implementar MySQL Master-Slave o migrar a Galera Cluster para permitir que el escalado horizontal de los microservicios sea efectivo. En el escenario ideal de producción, cada réplica del microservicio debería conectarse a una réplica de lectura de MySQL, reservando la instancia principal solo para escrituras.

Paginación en endpoints de alto volumen: El endpoint GET /api/citas debe implementar paginación mediante parámetros LIMIT y OFFSET para evitar la consulta completa de los 8.001 registros en cada solicitud. Esta mejora reduciría drásticamente la latencia y los errores bajo alta concurrencia sin necesidad de escalar la infraestructura.

Implementación de caché: Se recomienda incorporar Redis como capa de caché para los endpoints de consulta frecuente (servicios, barberos disponibles). Esto reduciría la carga sobre

las bases de datos MySQL y mejoraría significativamente los tiempos de respuesta bajo alta concurrencia.

Automatización del pipeline Spark: La ejecución manual del pipeline de análisis (exportar.py + spark-submit) debe automatizarse mediante un orquestador de workflows como Apache Airflow, programando la ejecución periódica (por ejemplo cada hora o al cierre del día) sin intervención manual del administrador.

Monitoreo y alertas: Se recomienda integrar un stack de monitoreo basado en Prometheus y Grafana para visualizar en tiempo real métricas de CPU, memoria, latencia y tasa de errores de cada microservicio, configurando alertas automáticas cuando los indicadores superen umbrales críticos.

Rate limiting en HAProxy: Para proteger el sistema contra picos de demanda no controlados, se recomienda configurar límites de tasa de solicitudes por IP en HAProxy, rechazando solicitudes que superen el umbral definido y retornando códigos HTTP 429 (Too Many Requests).

Escalado automático: En un entorno de producción con mayor capacidad de infraestructura, se recomienda migrar de Docker Swarm a Kubernetes para aprovechar las capacidades de Horizontal Pod Autoscaler (HPA), que escala automáticamente el número de réplicas en función de métricas como el uso de CPU o el tiempo de respuesta promedio.

8. REFERENCIAS

- [1] Docker Inc., "Docker Swarm overview," Docker Documentation, 2024. [Online]. Available: <https://docs.docker.com/engine/swarm/>
- [2] HAProxy Technologies, "HAProxy Documentation," 2024. [Online]. Available: <https://www.haproxy.org/>
- [3] Apache Software Foundation, "Apache Spark 3.5.8 Documentation," 2024. [Online]. Available: <https://spark.apache.org/docs/3.5.8/>
- [4] M. Fowler y J. Lewis, "Microservices," martinowler.com, 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [5] Apache Software Foundation, "Apache JMeter User's Manual," 2024. [Online]. Available: <https://jmeter.apache.org/usermanual/index.html>
- [6] HashiCorp, "Vagrant Documentation," 2024. [Online]. Available: <https://developer.hashicorp.com/vagrant/docs>
- [7] Oracle Corporation, "MySQL 8.0 Reference Manual," 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>

[8] Node.js Foundation, "Node.js Documentation," 2024. [Online]. Available: <https://nodejs.org/en/docs>

[9] M. Kleppmann, "Designing Data-Intensive Applications," O'Reilly Media, 2017.

[10] B. Burns, B. Grant, D. Oppenheimer, E. Brewer y J. Wilkes, "Borg, Omega, and Kubernetes," ACM Queue, vol. 14, 2016.